Технологии конструирования программного обеспечения

Отчет по лабораторной работе № 7

Группа: 241-327 Студент: Дубина Анастасия Александровна

Задание на лабораторную работу

Тема проекта: приложение «Журнал учителя». В приложении должно быть реализовано 2 класса (соответственно основной и дополнительный, основной класс содержит объекты дополнительного): Журнал и Урок. Класс Журнал содержит название предмета, имя учителя. Класс Урок содержит дату, тему, номер класса. Наследники дополнительного класса: Зачет, Экзамен.

Диаграмма классов

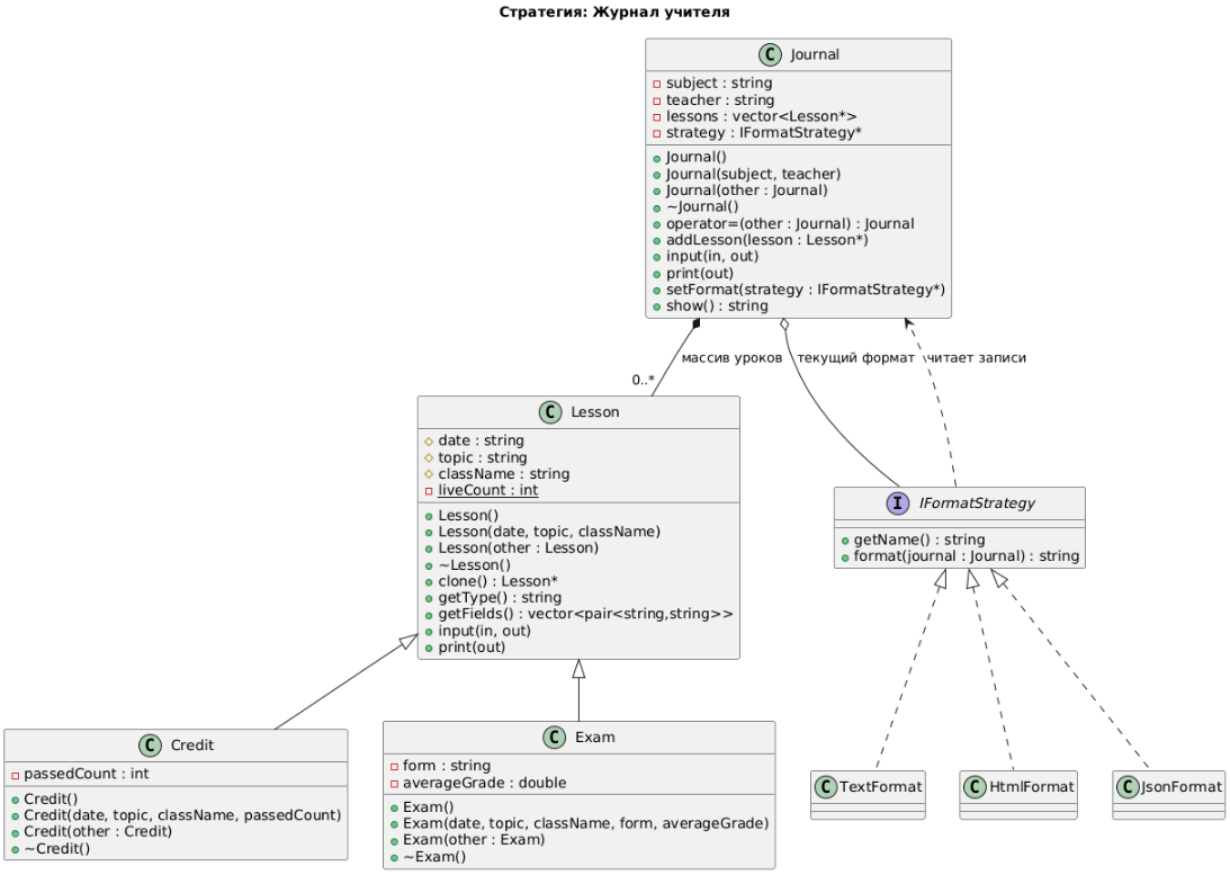


Диаграмма последовательности

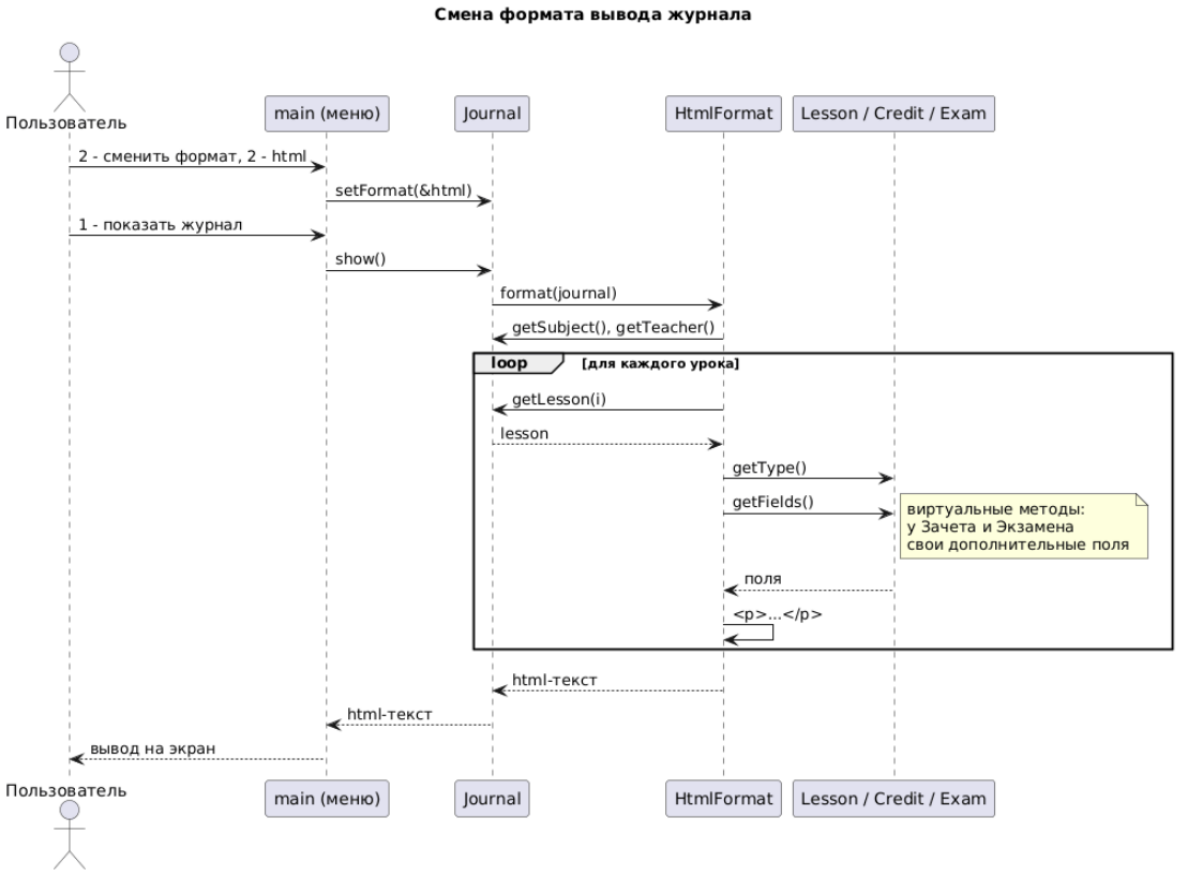
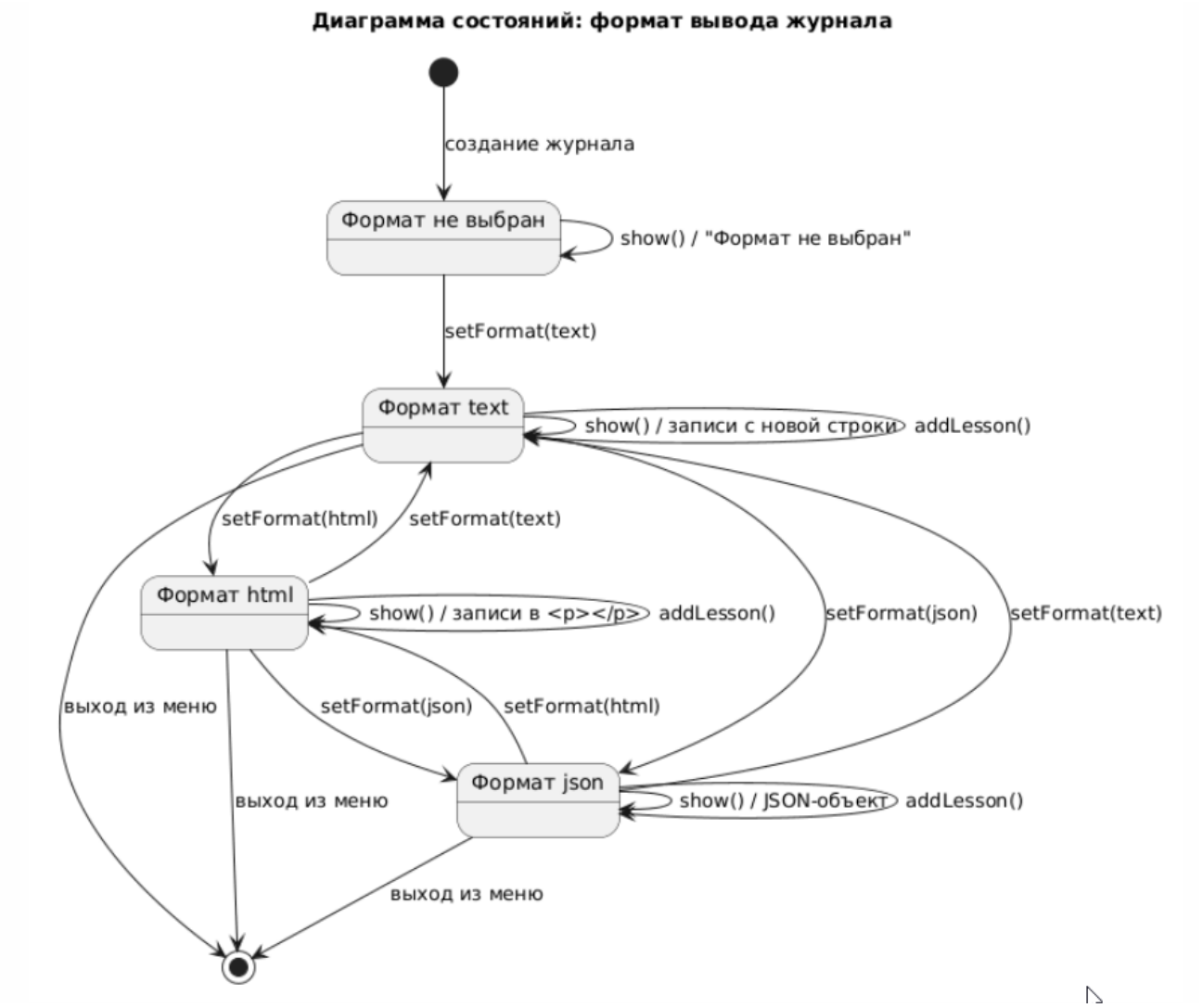


Диаграмма состояний



Исходный код программы

```
#include <iostream>
#include <sstream>
#include <iomanip>
#include <string>
#include <vector>
#include <utility>
#include <clocale>

using namespace std;

// ======================== Ввод данных ========================

string readLine(istream& in, ostream& out, const string& prompt) {
   out << prompt;
   string line;
   getline(in, line);
```

```
    if (!line.empty() && line.back() == '\r') line.pop_back();
    return line;
}

// Число можно ввести и через точку, и через запятую: "4.5" или "4,5".
// Разбор не зависит от setlocale (иначе "4.5" в русской локали читается как 4).
double readNumber(istream& in, ostream& out, const string& prompt) {
    string line = readLine(in, out, prompt);
    for (char& c : line) {
        if (c == ',') c = '.';
    }
    istringstream parser(line);
    parser.imbue(locale::classic());
    double value = 0;
    if (!(parser >> value)) return 0;
    return value;
}

// ======================== Дополнительный класс: Урок ========================

class Lesson {
public:
    // Конструктор без параметров
    Lesson() : date("01.09.2026"), topic("Без темы"), className("1А") {
        liveCount++;
    }

    // Конструктор с параметрами
    Lesson(const string& date, const string& topic, const string& className)
        : date(date), topic(topic), className(className) {
        liveCount++;
    }

    // Конструктор копирования
    Lesson(const Lesson& other)
        : date(other.date), topic(other.topic), className(other.className) {
        liveCount++;
    }

    // Деструктор (виртуальный, т.к. объекты удаляются через указатель на базовый класс)
    virtual ~Lesson() {
        liveCount--;
    }

    // Копия объекта с сохранением его настоящего типа (нужна для копирования журнала)
    virtual Lesson* clone() const { return new Lesson(*this); }
```

```
    virtual string getType() const { return "Урок"; }

    // Поля записи в виде пар «название - значение». По ним работают стратегии вывода.
    virtual vector<pair<string, string>> getFields() const {
        return { { "Дата", date }, { "Тема", topic }, { "Класс", className } };
    }

    // Ввод данных
    virtual void input(istream& in, ostream& out) {
        date = readLine(in, out, "Дата: ");
        topic = readLine(in, out, "Тема: ");
        className = readLine(in, out, "Класс: ");
    }

    // Вывод данных
    void print(ostream& out) const {
        out << getType();
        for (const auto& field : getFields()) {
            out << " | " << field.first << ": " << field.second;
        }
        out << endl;
    }

    string getDate() const { return date; }
    string getTopic() const { return topic; }
    string getClassName() const { return className; }

    // Сколько объектов Урок сейчас существует (для проверки деструкторов)
    static int getLiveCount() { return liveCount; }

protected:
    string date;
    string topic;
    string className;

private:
    static int liveCount;
};

int Lesson::liveCount = 0;

// ======================== Наследники Урока ========================

class Credit : public Lesson {
public:
    Credit() : Lesson(), passedCount(0) {}
```

```
    Credit(const string& date, const string& topic, const string& className, int passedCount)
        : Lesson(date, topic, className), passedCount(passedCount) {}

    Credit(const Credit& other) : Lesson(other), passedCount(other.passedCount) {}

    ~Credit() override {}

    Lesson* clone() const override { return new Credit(*this); }

    string getType() const override { return "Зачет"; }

    vector<pair<string, string>> getFields() const override {
        vector<pair<string, string>> fields = Lesson::getFields();
        fields.push_back({ "Сдали", to_string(passedCount) + " чел." });
        return fields;
    }

    void input(istream& in, ostream& out) override {
        Lesson::input(in, out);
        passedCount = (int)readNumber(in, out, "Сколько учеников сдали: ");
    }

    int getPassedCount() const { return passedCount; }

private:
    int passedCount;
};

class Exam : public Lesson {
public:
    Exam() : Lesson(), form("письменный"), averageGrade(0) {}

    Exam(const string& date, const string& topic, const string& className,
         const string& form, double averageGrade)
        : Lesson(date, topic, className), form(form), averageGrade(averageGrade) {}

    Exam(const Exam& other) : Lesson(other), form(other.form), averageGrade(other.averageGrade) {}

    ~Exam() override {}

    Lesson* clone() const override { return new Exam(*this); }

    string getType() const override { return "Экзамен"; }

    vector<pair<string, string>> getFields() const override {
        vector<pair<string, string>> fields = Lesson::getFields();
        ostringstream grade;
```

```
        grade << fixed << setprecision(1) << averageGrade;
        fields.push_back({ "Форма", form });
        fields.push_back({ "Средний балл", grade.str() });
        return fields;
    }

    void input(istream& in, ostream& out) override {
        Lesson::input(in, out);
        form = readLine(in, out, "Форма (устный/письменный): ");
        averageGrade = readNumber(in, out, "Средний балл: ");
    }

    string getForm() const { return form; }
    double getAverageGrade() const { return averageGrade; }

private:
    string form;
    double averageGrade;
};

// ======================== Стратегия (интерфейс) ========================

class Journal;

class IFormatStrategy {
public:
    virtual ~IFormatStrategy() {}
    virtual string getName() const = 0;
    virtual string format(const Journal& journal) const = 0;
};

// ======================== Основной класс: Журнал (контекст) ========================

class Journal {
public:
    Journal() : subject("Без предмета"), teacher("Не указан"), strategy(nullptr) {}

    Journal(const string& subject, const string& teacher)
        : subject(subject), teacher(teacher), strategy(nullptr) {}

    // Конструктор копирования: глубокая копия, каждый урок копируется через clone()
    Journal(const Journal& other)
        : subject(other.subject), teacher(other.teacher), strategy(other.strategy) {
        for (Lesson* lesson : other.lessons) {
            lessons.push_back(lesson->clone());
        }
    }
```

```
// Присваивание через копию и обмен
Journal& operator=(Journal other) {
   swap(subject, other.subject);
   swap(teacher, other.teacher);
   swap(strategy, other.strategy);
   swap(lessons, other.lessons);
   return *this;
}

// Деструктор: журнал владеет уроками и удаляет их
~Journal() {
   for (Lesson* lesson : lessons) {
      delete lesson;
   }
}

// Журнал становится владельцем переданного урока
void addLesson(Lesson* lesson) { lessons.push_back(lesson); }

int getCount() const { return (int)lessons.size(); }
const Lesson& getLesson(int index) const { return *lessons[index]; }

string getSubject() const { return subject; }
string getTeacher() const { return teacher; }

// Ввод данных журнала
void input(istream& in, ostream& out) {
   subject = readLine(in, out, "Предмет: ");
   teacher = readLine(in, out, "Учитель: ");
}

// Вывод данных журнала
void print(ostream& out) const {
   out << "Журнал: " << subject << ", учитель: " << teacher
      << ", записей: " << lessons.size() << endl;
   for (const Lesson* lesson : lessons) {
      out << "  ";
      lesson->print(out);
   }
}

// Смена стратегии во время работы программы
void setFormat(const IFormatStrategy* newStrategy) { strategy = newStrategy; }
const IFormatStrategy* getFormat() const { return strategy; }

string show() const {
```

```
        if (strategy == nullptr) return "Формат вывода не выбран\n";
        return strategy->format(*this);
    }

private:
    string subject;
    string teacher;
    vector<Lesson*> lessons;     // массив объектов базового класса Урок
    const IFormatStrategy* strategy;
};

// ======================== Конкретные стратегии ========================

// Каждая запись с новой строки
class TextFormat : public IFormatStrategy {
public:
    string getName() const override { return "text"; }

    string format(const Journal& journal) const override {
        ostringstream out;
        out << "Журнал: " << journal.getSubject() << " (учитель: " << journal.getTeacher() << ")\n";
        for (int i = 0; i < journal.getCount(); i++) {
            const Lesson& lesson = journal.getLesson(i);
            out << i + 1 << ". " << lesson.getType() << ":";
            for (const auto& field : lesson.getFields()) {
                out << " " << field.first << " - " << field.second << ";";
            }
            out << "\n";
        }
        return out.str();
    }
};

// Каждая запись обрамляется тегами <p></p>
class HtmlFormat : public IFormatStrategy {
public:
    string getName() const override { return "html"; }

    string format(const Journal& journal) const override {
        ostringstream out;
        out << "<h1>" << escape(journal.getSubject()) << "</h1>\n";
        out << "<h2>Учитель: " << escape(journal.getTeacher()) << "</h2>\n";
        for (int i = 0; i < journal.getCount(); i++) {
            const Lesson& lesson = journal.getLesson(i);
            out << "<p><b>" << escape(lesson.getType()) << "</b>";
            for (const auto& field : lesson.getFields()) {
                out << " " << escape(field.first) << ": " << escape(field.second) << ";";
```

```
            }
            out << "</p>\n";
        }
        return out.str();
    }

private:
    static string escape(const string& s) {
        string result;
        for (char c : s) {
            if (c == '<') result += "&lt;";
            else if (c == '>') result += "&gt;";
            else if (c == '&') result += "&amp;";
            else result += c;
        }
        return result;
    }
};

// Журнал в формате JSON
class JsonFormat : public IFormatStrategy {
public:
    string getName() const override { return "json"; }

    string format(const Journal& journal) const override {
        ostringstream out;
        out << "{\n";
        out << "  \"subject\": \"" << escape(journal.getSubject()) << "\",\n";
        out << "  \"teacher\": \"" << escape(journal.getTeacher()) << "\",\n";
        out << "  \"lessons\": [\n";
        for (int i = 0; i < journal.getCount(); i++) {
            const Lesson& lesson = journal.getLesson(i);
            out << "    { \"type\": \"" << escape(lesson.getType()) << "\"";
            for (const auto& field : lesson.getFields()) {
                out << ", \"" << escape(field.first) << "\": \"" << escape(field.second) << "\"";
            }
            out << " }" << (i + 1 < journal.getCount() ? "," : "") << "\n";
        }
        out << "  ]\n}\n";
        return out.str();
    }

private:
    static string escape(const string& s) {
        string result;
        for (char c : s) {
            if (c == '"') result += "\\\"";
```

```
            else if (c == '\\') result += "\\\\";
            else result += c;
        }
        return result;
    }
};

// ======================== Тестирование ========================

class Tester {
public:
    Tester() : passed(0), failed(0) {}

    void check(bool condition, const string& name) {
        if (condition) {
            passed++;
            cout << "  [OK]   " << name << endl;
        } else {
            failed++;
            cout << "  [FAIL] " << name << endl;
        }
    }

    void printResult() const {
        cout << "Пройдено: " << passed << ", провалено: " << failed << endl;
    }

private:
    int passed;
    int failed;
};

bool contains(const string& text, const string& part) {
    return text.find(part) != string::npos;
}

int countOf(const string& text, const string& part) {
    int count = 0;
    for (size_t pos = text.find(part); pos != string::npos; pos = text.find(part, pos + 1)) count++;
    return count;
}

Journal makeJournal() {
    Journal journal("Математика", "Иванова М.П.");
    journal.addLesson(new Lesson("02.09.2026", "Дроби", "5А"));
    journal.addLesson(new Credit("20.10.2026", "Уравнения", "5А", 24));
    journal.addLesson(new Exam("25.12.2026", "Итоги полугодия", "5А", "письменный", 4.2));
```

```
    return journal;
}

void testConstructors(Tester& t) {
    Lesson empty;
    t.check(empty.getTopic() == "Без темы" && empty.getClassName() == "1А", "Конструктор Урока без
параметров");

    Exam exam("25.12.2026", "Итоги", "9Б", "устный", 3.8);
    Exam copy(exam);
    t.check(copy.getTopic() == "Итоги" && copy.getForm() == "устный" && copy.getAverageGrade() == 3.8,
        "Конструктор копирования Экзамена");

    Credit credit;
    t.check(credit.getPassedCount() == 0 && credit.getType() == "Зачет", "Конструктор Зачета без параметров");

    Journal journal;
    t.check(journal.getSubject() == "Без предмета" && journal.getCount() == 0, "Конструктор Журнала без
параметров");
}

void testInput(Tester& t) {
    ostringstream silent;
    istringstream in("10.11.2026\nТреугольники\n7В\nустный\n4.5\n");
    Exam exam;
    exam.input(in, silent);
    t.check(exam.getDate() == "10.11.2026" && exam.getTopic() == "Треугольники" &&
        exam.getClassName() == "7В" && exam.getForm() == "устный" && exam.getAverageGrade() == 4.5,
        "Ввод данных Экзамена");

    ostringstream out;
    exam.print(out);
    t.check(contains(out.str(), "Экзамен") && contains(out.str(), "Треугольники"), "Вывод данных Экзамена");

    istringstream journalIn("Физика\nПетров А.А.\n");
    Journal journal;
    journal.input(journalIn, silent);
    t.check(journal.getSubject() == "Физика" && journal.getTeacher() == "Петров А.А.", "Ввод данных Журнала");
}

void testPolymorphicArray(Tester& t) {
    Journal journal = makeJournal();
    t.check(journal.getCount() == 3 &&
        journal.getLesson(0).getType() == "Урок" &&
        journal.getLesson(1).getType() == "Зачет" &&
        journal.getLesson(2).getType() == "Экзамен",
        "Массив базового типа хранит объекты базового класса и наследников");
```

```
}

void testCopyAndDestructors(Tester& t) {
   int before = Lesson::getLiveCount();
   {
      Journal original = makeJournal();
      Journal copy(original);
      copy.addLesson(new Lesson("01.10.2026", "Проценты", "5А"));
      t.check(original.getCount() == 3 && copy.getCount() == 4, "Копия журнала независима от оригинала");
      t.check(&original.getLesson(0) != &copy.getLesson(0) && copy.getLesson(2).getType() == "Экзамен",
            "Уроки копируются глубоко с сохранением типа");

      Journal assigned;
      assigned = original;
      t.check(assigned.getCount() == 3 && assigned.getSubject() == "Математика", "Оператор присваивания");
   }
   t.check(Lesson::getLiveCount() == before, "Деструкторы удалили все уроки");
}

void testStrategies(Tester& t) {
   Journal journal = makeJournal();
   TextFormat text;
   HtmlFormat html;
   JsonFormat json;

   t.check(contains(journal.show(), "не выбран"), "Без стратегии выводится предупреждение");

   journal.setFormat(&text);
   string textOut = journal.show();
   t.check(contains(textOut, "1. Урок") && contains(textOut, "3. Экзамен"), "Формат text: каждая запись с новой
строки");

   journal.setFormat(&html);
   string htmlOut = journal.show();
   t.check(countOf(htmlOut, "<p>") == 3 && countOf(htmlOut, "</p>") == 3, "Формат html: каждая запись в
<p></p>");

   journal.setFormat(&json);
   string jsonOut = journal.show();
   t.check(jsonOut[0] == '{' && contains(jsonOut, "\"type\": \"Зачет\"") && contains(jsonOut, "\"Сдали\": \"24
чел.\""),
         "Формат json");

   t.check(textOut != htmlOut && htmlOut != jsonOut, "Смена стратегии меняет вывод того же журнала");
}

void runTests() {
```

```
    Tester t;
    cout << "\n===== Тестирование =====" << endl;
    testConstructors(t);
    testInput(t);
    testPolymorphicArray(t);
    testCopyAndDestructors(t);
    testStrategies(t);
    t.printResult();
}

// ====================== Меню ======================

void addLessonFromUser(Journal& journal) {
    string type = readLine(cin, cout, "Тип записи (1 - урок, 2 - зачет, 3 - экзамен): ");
    Lesson* lesson;
    if (type == "2") lesson = new Credit();
    else if (type == "3") lesson = new Exam();
    else lesson = new Lesson();
    lesson->input(cin, cout);
    journal.addLesson(lesson);
    cout << "Запись добавлена" << endl;
}

void runMenu(Journal& journal) {
    TextFormat text;
    HtmlFormat html;
    JsonFormat json;
    journal.setFormat(&text);

    while (true) {
        cout << "\nТекущий формат: " << journal.getFormat()->getName() << endl;
        cout << "1 - показать журнал, 2 - сменить формат, 3 - добавить запись, 0 - выход" << endl;
        string choice = readLine(cin, cout, "> ");
        if (!cin || choice == "0") break;

        if (choice == "1") {
            cout << journal.show();
        } else if (choice == "2") {
            string f = readLine(cin, cout, "Формат (1 - text, 2 - html, 3 - json): ");
            if (f == "1") journal.setFormat(&text);
            else if (f == "2") journal.setFormat(&html);
            else if (f == "3") journal.setFormat(&json);
            else cout << "Неизвестный формат" << endl;
        } else if (choice == "3") {
            addLessonFromUser(journal);
        } else {
            cout << "Неизвестная команда" << endl;
```

```
        }
    }
    journal.setFormat(nullptr);
}

// ======================== Демонстрация ======================

int main() {
    setlocale(LC_ALL, "Russian");

    Journal journal = makeJournal();
    journal.addLesson(new Credit("15.11.2026", "Геометрия", "5А", 22));

    cout << "===== Демонстрация =====" << endl;
    cout << "--- Метод вывода print() ---" << endl;
    journal.print(cout);

    TextFormat text;
    HtmlFormat html;
    JsonFormat json;
    vector<const IFormatStrategy*> strategies = { &text, &html, &json };
    for (const IFormatStrategy* s : strategies) {
        journal.setFormat(s);
        cout << "\n--- Формат " << s->getName() << " ---" << endl;
        cout << journal.show();
    }
    journal.setFormat(nullptr);

    runTests();

    cout << "\n===== Меню =====" << endl;
    runMenu(journal);
    return 0;
}
```